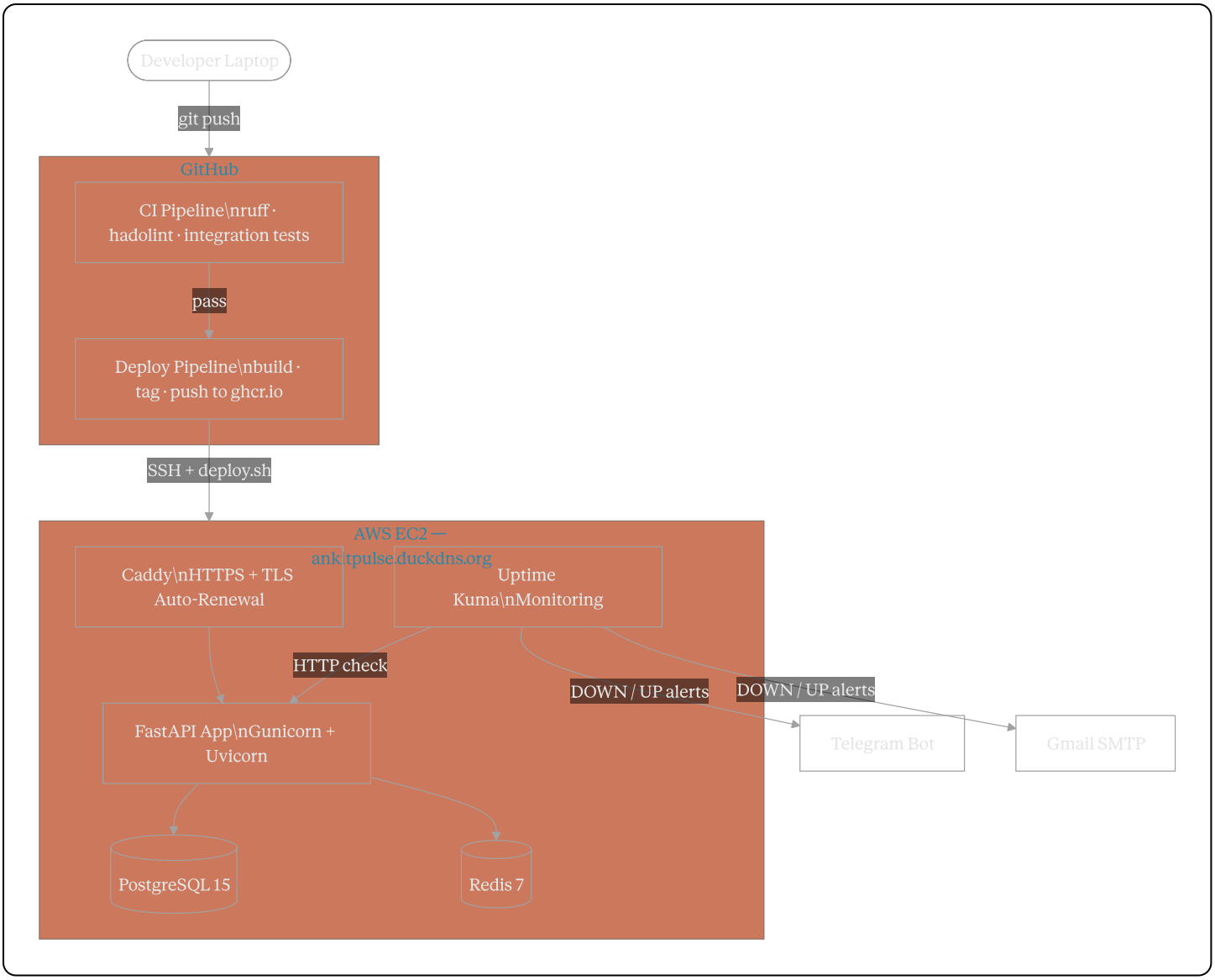
StatusPulse

> A production-ready status page and health monitoring API — deployed with full CI/CD, HTTPS, and observability.

Live URL	https://ankitpulse.duckdns.org
Health Check	https://ankitpulse.duckdns.org/health
API Docs	https://ankitpulse.duckdns.org/docs
Monitoring Dashboard	http://98.89.150.237:3001
GitHub	https://github.com/Ankitkhan08/statuspulse

Table of Contents

Architecture



Tech Stack

Layer	Technology
API Framework	FastAPI (Python 3.11)
Database	PostgreSQL 15 Alpine
Cache / PubSub	Redis 7 Alpine
WSGI Server	Gunicorn + Uvicorn workers
Reverse Proxy	Caddy (automatic Let's Encrypt TLS)
Containerization	Docker + Docker Compose
CI/CD	GitHub Actions

Layer	Technology
Container Registry	GitHub Container Registry (ghcr.io)
Cloud Provider	AWS EC2 (Ubuntu)
IaC	Ansible
Monitoring	Uptime Kuma
Alerting	Telegram Bot API + Gmail SMTP
Security Scanning	Trivy

Prerequisites

- Docker Desktop
- Git
- GitHub account
- An Ubuntu 22.04+ server (for production)

Local Development

1. Clone the repository

```bash
git clone https://github.com/Ankitkhan08/statuspulse.git
cd statuspulse
```

2. Configure environment variables

```bash
cp .env.example .env
```

Open `.env` and set your passwords:

```env
```

```
DB_NAME=statuspulse
DB_USER=statuspulse_user
DB_PASSWORD=your_strong_password
REDIS_PASSWORD=your_redis_password
APP_PORT=8000
```

3. Start the full stack

bash

```
docker compose up -d
```

The app waits for PostgreSQL and Redis to pass their health checks before starting. Allow ~20 seconds on first run.

4. Verify health

bash

```
curl http://localhost:8000/health
```

Expected response:

json

```
{
  "status": "healthy",
  "checks": {
    "api": "healthy",
    "database": "healthy",
    "redis": "healthy"
  },
  "timestamp": "2026-05-11T10:35:58+00:00"
}
```

5. Browse the API

Open http://localhost:8000/docs for the interactive Swagger UI.

Makefile Commands

bash

```
make build    # Build the Docker image
make up       # Start all services (app + postgres + redis)
make down     # Stop all services
make logs     # Tail logs from all services
make test     # Run health check via curl
make clean    # Remove containers, images, and volumes
make shell    # Open bash inside the running app container
```

CI/CD Pipeline

Continuous Integration — `.github/workflows/ci.yml`

Triggered on every push and pull request to `main`.

Step	Tool	Purpose
Python Lint	`ruff`	Enforce code style
Dockerfile Lint	`hadolint`	Catch Dockerfile best-practice violations
Image Build	Docker	Verify the image builds successfully
Stack Start	Docker Compose	Spin up the full app + db + cache stack
Integration Tests	`tests/test_integration.sh`	Validate all API endpoints end-to-end
Artifact Upload	GitHub Actions	Persist test results for review
Stack Teardown	Docker Compose	Clean up runner environment

Continuous Deployment — `.github/workflows/deploy.yml`

Triggered on every push to `main`.

1. Build the Docker image and tag it with `latest` and the commit SHA.
2. Push both tags to `ghcr.io/ankitkhan08/statuspulse`.
3. SSH into the AWS EC2 server using GitHub Secrets.
4. Execute `scripts/deploy.sh` on the server.
5. The script verifies `/health` and **automatically rolls back** if the check fails.

Security Scan — `.github/workflows/security.yml`

Triggered on every push to `main`. Runs Trivy against the built image and reports `CRITICAL` and `HIGH` CVEs directly in the Actions log.

Integration Test Coverage — `tests/test_integration.sh`

Endpoint	Method	Assertion
`/health`	GET	HTTP 200 · all checks healthy
`/`	GET	HTTP 200 · service name present
`/services`	POST	HTTP 200 · `id` returned
`/services` (duplicate)	POST	HTTP 409 · conflict handled
`/services`	GET	HTTP 200 · array contains entry
`/incidents`	POST	HTTP 200 · status `investigating`
`/incidents`	GET	HTTP 200 · array contains entry

Production Deployment

Infrastructure Overview

- **Server:** AWS EC2 (Ubuntu 22.04, t2.micro)
- **Domain:** `ankitpulse.duckdns.org` (DuckDNS free tier)
- **HTTPS:** Caddy with automatic Let's Encrypt certificate
- **Firewall:** UFW — ports 22, 80, 443, 3001 only
- **IaC:** Ansible playbook manages server state idempotently

Manual Deployment Steps

bash

```
# 1. SSH into the server
ssh -i ~/.ssh/your-key.pem ubuntu@YOUR_SERVER_IP

# 2. Clone the repository
git clone https://github.com/Ankitkhan08/statuspulse.git
cd statuspulse

# 3. Configure environment
cp .env.example .env
nano .env   # set production-grade passwords

# 4. Start the application stack
docker compose up -d

# 5. Configure Caddy for HTTPS
sudo tee /etc/caddy/Caddyfile << 'EOF'
{
    email your@email.com
}

yourdomain.duckdns.org {
    header Strict-Transport-Security "max-age=31536000; includeSubDomains"
    header X-Content-Type-Options "nosniff"
    header X-Frame-Options "DENY"
    header X-XSS-Protection "1; mode=block"
    reverse_proxy localhost:8000
    encode gzip
}
EOF
sudo systemctl restart caddy
```

Automated Deployment via `scripts/deploy.sh`

The deploy script is invoked automatically by the CD pipeline. It can also be run manually:

bash

```
bash ~/statuspulse/scripts/deploy.sh
```

What it does:

1. Pulls the latest image from `ghcr.io`
2. Restarts the stack with the new image
3. Polls `/health` up to 10 times (5-second intervals)
4. Exits 0 on success — exits 1 and rolls back on failure

Server Hardening (via Ansible)

```bash
# Run the Ansible playbook (idempotent — safe to run repeatedly)
sudo ansible-playbook -c local -i localhost, ansible/playbook.yml
```

The playbook enforces:

- UFW firewall rules
- 1 GB swap file for memory safety
- Daily PostgreSQL backup cron job

Monitoring & Alerting

Uptime Kuma

Accessible at http://98.89.150.237:3001

Monitor	Type	Interval
StatusPulse API	HTTP — `/health` endpoint	60 seconds
TLS Certificate	SSL expiry check	Daily

Alert channels configured: Telegram Bot + Gmail SMTP

Both channels receive **DOWN** alerts (with recovery time) and **UP** alerts when the service recovers.

System Health Monitor — `scripts/health-monitor.sh`

A cron job runs every 5 minutes and checks:

- `/health` endpoint reachability
- Disk usage — alerts if above 90%
- Memory usage — alerts if above 90%
- All expected Docker containers are running

Alerts are pushed to Telegram and logged locally:

```bash
```

```
# View the cron schedule
crontab -l

# Stream live logs
tail -f /var/log/statuspulse-monitor.log
```

Backup & Restore

How backups work

`scripts/backup.sh` runs `pg_dump` inside the PostgreSQL container, compresses the output with gzip, and saves it to `/home/ubuntu/backups/`. Backups older than 7 days are automatically removed.

bash

```
# Run a manual backup
bash scripts/backup.sh

# List existing backups
ls -lh ~/backups/
```

The daily backup cron (2:00 AM) is configured by Ansible.

Restore from backup

bash

```
# Replace the filename with your target backup
gunzip < ~/backups/pg_backup_YYYY-MM-DD_HHMMSS.sql.gz | \
  docker exec -i statuspulse_postgres \
  psql -U statuspulse_user -d statuspulse
```

Security

Area	Implementation
Container runtime	Non-root user `appuser` (UID 1001)
Image size	Multi-stage build — final image under 85 MB
Secrets at rest	`.env` excluded via `.gitignore`

Area	Implementation
Secrets in CI/CD	GitHub Actions encrypted secrets
Firewall	UFW — deny all except 22, 80, 443, 3001
SSH	`PermitRootLogin no` · `PasswordAuthentication no`
TLS	Caddy + Let's Encrypt — auto-renewed
Security headers	HSTS · X-Frame-Options · X-Content-Type-Options · X-XSS-Protection
Vulnerability scanning	Trivy integrated in CI pipeline (`security.yml`)
Internal networking	PostgreSQL and Redis isolated in `statuspulse_net` Docker network — not publicly exposed

Troubleshooting

App container keeps restarting:

```bash
docker compose logs app
docker compose down && docker compose up -d
```

Database connection errors:

```bash
docker exec statuspulse_postgres pg_isready -U statuspulse_user
cat .env   # verify DB_HOST, DB_USER, DB_PASSWORD
```

HTTPS not working / certificate issues:

```bash
sudo journalctl -u caddy -n 30
sudo systemctl restart caddy
```

SSH connection refused:

```bash
```

```
# On server console
sudo ufw status
sudo systemctl status ssh
```

CI pipeline failing:

- Check the Actions tab on GitHub for the failing step log
- Ensure `app/main.py` has no syntax errors
- Verify `.github/workflows/ci.yml` YAML indentation is correct

Repository Structure

```
statuspulse/
├── .github/
│   └── workflows/
│       ├── ci.yml           # Lint · build · test
│       ├── deploy.yml       # Push to ghcr.io · SSH deploy
│       └── security.yml     # Trivy vulnerability scan
├── ansible/
│   ├── playbook.yml         # Server configuration (IaC)
│   └── inventory.ini        # Target host definition
├── app/
│   ├── main.py              # FastAPI application (provided — do not modify)
│   └── requirements.txt     # Python dependencies
├── caddy/
│   └── Caddyfile            # Reverse proxy + HTTPS config
├── scripts/
│   ├── deploy.sh            # Zero-downtime deploy + rollback
│   ├── backup.sh            # PostgreSQL backup + rotation
│   └── health-monitor.sh    # System health cron script
├── tests/
│   └── test_integration.sh # End-to-end API test suite
├── screenshots/             # Proof screenshots for each task
├── Dockerfile               # Multi-stage production build
├── docker-compose.yml       # Local + production orchestration
├── Makefile                 # Developer convenience commands
├── .env.example             # Environment variable template
├── .gitignore
├── README.md                # This file
└── SECURITY.md              # Security documentation
```
